

Testing and Continuous Integration

Emad Jacob Maroun

Technical University of Denmark
Embedded Systems Engineering

September 30, 2026

Agile Manifesto Focus

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan ¹

¹<https://agilemanifesto.org/>

Agile Manifesto Focus

Individuals and **interactions** over processes and tools

Working software ~~software~~ **hardware** over comprehensive
documentation

Customer collaboration over contract negotiation

Responding to change over following a plan ¹

¹ <https://agilemanifesto.org/>

Outline

- ▶ What testing is, and what it is for
- ▶ Running example: a FIFO queue
- ▶ Testing levels and test-driven development
- ▶ Property-based testing with ScalaCheck
- ▶ Continuous integration with GitHub Actions
- ▶ Lab: a priority list

Testing vs. Verification

- ▶ Testing: does the design do what I meant?
 - ▶ Written and run by the developer
 - ▶ Small and fast, so it fits inside the edit-run loop

Testing vs. Verification

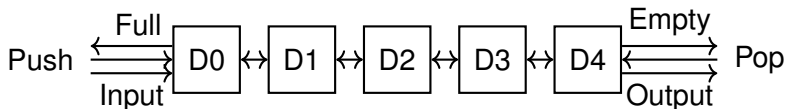
- ▶ Testing: does the design do what *I* meant?
 - ▶ Written and run by the developer
 - ▶ Small and fast, so it fits inside the edit-run loop
- ▶ Verification: does the design do what the *specification* says?
 - ▶ A bigger effort, ideally done by someone other than the developer
 - ▶ Formal verification comes later in the course

Testing vs. Verification

- ▶ Testing: does the design do what *I* meant?
 - ▶ Written and run by the developer
 - ▶ Small and fast, so it fits inside the edit-run loop
- ▶ Verification: does the design do what the *specification* says?
 - ▶ A bigger effort, ideally done by someone other than the developer
 - ▶ Formal verification comes later in the course
- ▶ Why test at all?
 - ▶ Find bugs while the code is still fresh in your head
 - ▶ Find regressions later, when it is not
 - ▶ Tests document what a module promises
 - ▶ A shared test list is how a team agrees on what “done” means

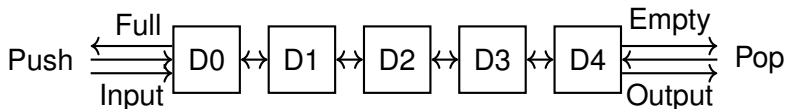
Running Example: FIFO Queue

- ▶ First-In, First-Out: the first item pushed is the first item popped
- ▶ Sits between a producer and a consumer that do not run in lockstep, for example on either side of a clock domain crossing
- ▶ Ours is a chain of stages; an item ripples forward one stage per cycle



Running Example: FIFO Queue

- ▶ First-In, First-Out: the first item pushed is the first item popped
- ▶ Sits between a producer and a consumer that do not run in lockstep, for example on either side of a clock domain crossing
- ▶ Ours is a chain of stages; an item ripples forward one stage per cycle



Six ports: push, input and full on one side, pop, output and empty on the other

Testing Levels

1. **Unit tests:** one component in isolation
2. **Integration tests:** components working together
3. **System tests:** the complete product, driven by use cases
4. **Acceptance tests:** run by the client
5. **Non-functional tests:** timing, area, power

Testing Levels

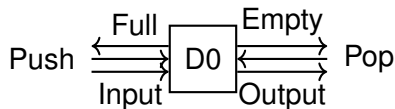
1. **Unit tests:** one component in isolation
 - ▶ FIFO: a single stage
2. **Integration tests:** components working together
 - ▶ FIFO: the whole chain
3. **System tests:** the complete product, driven by use cases
 - ▶ FIFO: buffering inside the lab's priority list
4. **Acceptance tests:** run by the client
 - ▶ The TA checks your lab
5. **Non-functional tests:** timing, area, power
 - ▶ Does it still fit the FPGA and meet the clock?

Testing Levels - Unit Tests

- ▶ Pick one small feature
- ▶ Write tests that pin it down
- ▶ Run locally, before pushing

Testing Levels - Unit Tests

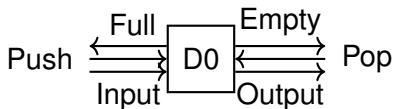
- ▶ Pick one small feature
- ▶ Write tests that pin it down
- ▶ Run locally, before pushing



FIFO Unit Tests (10): Exercise: 3 mins

Testing Levels - Unit Tests

- ▶ Pick one small feature
- ▶ Write tests that pin it down
- ▶ Run locally, before pushing



FIFO Unit Tests (10): Exercise: 3 mins

- ▶ Initialized to empty
- ▶ On push becomes full & !empty
- ▶ On pop becomes !full & empty
- ▶ If empty, pop does nothing (pops zero?)
- ▶ If empty, push updates next pop
- ▶ If full, can pop
- ▶ If full, push doesn't change next pop
- ▶ Values pushed will be popped next cycle
- ▶ Can store indefinitely
- ▶ Can push and pop simultaneously (forwarding?)

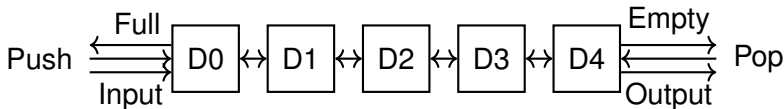
Testing Levels - Integration Tests

- ▶ Pick a common interaction between modules
- ▶ Write tests that pin it down
- ▶ Run by developers or dedicated testers

Testing Levels - Integration Tests

- ▶ Pick a common interaction between modules
- ▶ Write tests that pin it down
- ▶ Run by developers or dedicated testers

FIFO Tests (9): Exercise: 5 mins

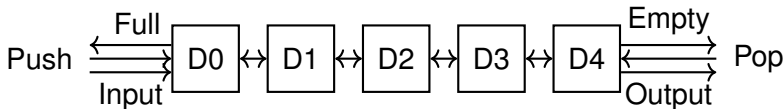


Testing Levels - Integration Tests

- ▶ Pick a common interaction between modules
- ▶ Write tests that pin it down
- ▶ Run by developers or dedicated testers

FIFO Tests (9): Exercise: 5 mins

- ▶ Initialized to empty
- ▶ On push becomes !empty
- ▶ On pop becomes !full
- ▶ Pushed item will pop eventually
- ▶ List of items will pop in pushed order
- ▶ If full, can pop and push simultaneously
- ▶ Can store indefinitely
- ▶ Items get to the front without popping
- ▶ Two lists pushed eventually become one list



Test-Driven Development (TDD)

- ▶ Write the test **before** the hardware
- ▶ Short cycle: **Red** → **Green** → **Refactor**

Test-Driven Development (TDD)

- ▶ Write the test **before** the hardware
- ▶ Short cycle: **Red** → **Green** → **Refactor**

Take the first item from your list: *initialized to empty*

```
"A stage" should "start empty" in {  
  test(new FifoStage(8)) { c =>  
    c.io.empty.expect(true.B)  
    c.io.full.expect(false.B)  
  }  
}
```

Test-Driven Development (TDD)

- ▶ Write the test **before** the hardware
- ▶ Short cycle: **Red** → **Green** → **Refactor**

Take the first item from your list: *initialized to empty*

```
"A stage" should "start empty" in {  
  test(new FifoStage(8)) { c =>  
    c.io.empty.expect(true.B)  
    c.io.full.expect(false.B)  
  }  
}
```

1. Run it. It does not even compile, there is no `FifoStage` yet (**red**)
2. Write just enough hardware: one register, initialised to empty (**green**)
3. Clean up, run again, take the next item from the list

Designing for Testability

- ▶ A test can only control the inputs and observe the outputs
- ▶ From the ports of the FIFO you cannot see what is in D2

Designing for Testability

- ▶ A test can only control the inputs and observe the outputs
- ▶ From the ports of the FIFO you cannot see what is in D2
- ▶ So how do you test “items get to the front without popping”?

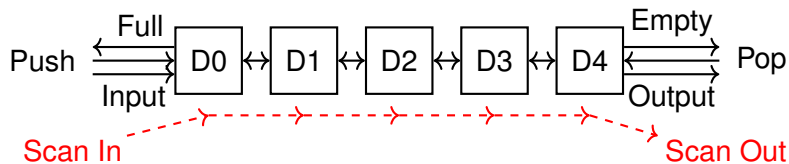
Designing for Testability

- ▶ A test can only control the inputs and observe the outputs
- ▶ From the ports of the FIFO you cannot see what is in D2
- ▶ So how do you test “items get to the front without popping”?
 - ▶ Wait long enough, then pop and check: slow and indirect
 - ▶ Add a debug port that exposes the stages: costs hardware, only in test builds?
 - ▶ Reuse the stages as one long shift register: a **scan chain**

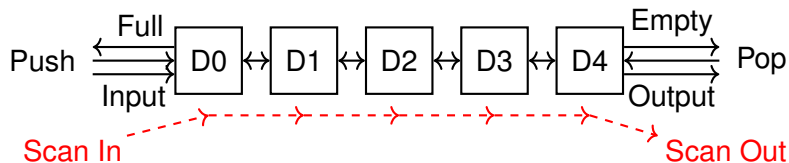
Designing for Testability

- ▶ A test can only control the inputs and observe the outputs
- ▶ From the ports of the FIFO you cannot see what is in D2
- ▶ So how do you test “items get to the front without popping”?
 - ▶ Wait long enough, then pop and check: slow and indirect
 - ▶ Add a debug port that exposes the stages: costs hardware, only in test builds?
 - ▶ Reuse the stages as one long shift register: a **scan chain**
- ▶ The general rule: small modules with clear ports are the easy ones to test
- ▶ Keeping control and data paths apart helps, so does keeping all stages identical

Scan Chain - FIFO



Scan Chain - FIFO



How does the scan chain change testing?

Property-Based Testing (PBT)

- ▶ Your unit tests poke a handful of values that you picked by hand
- ▶ The input space of even a small FIFO is enormous: which values, how many, in which order?

Property-Based Testing (PBT)

- ▶ Your unit tests poke a handful of values that you picked by hand
- ▶ The input space of even a small FIFO is enormous: which values, how many, in which order?
- ▶ PBT turns it around: state a **property** that must hold for *all* inputs, and let a tool pick the inputs
 1. Name the property: “push then pop returns the value”
 2. Describe the valid inputs with a **generator**
 3. Run the property a few hundred times with random inputs

Property-Based Testing (PBT)

- ▶ Your unit tests poke a handful of values that you picked by hand
- ▶ The input space of even a small FIFO is enormous: which values, how many, in which order?
- ▶ PBT turns it around: state a **property** that must hold for *all* inputs, and let a tool pick the inputs
 1. Name the property: “push then pop returns the value”
 2. Describe the valid inputs with a **generator**
 3. Run the property a few hundred times with random inputs
- ▶ Properties about order need an expected answer. A Scala `List` is a fine reference model for a FIFO

Property-Based Testing (PBT)

- ▶ Your unit tests poke a handful of values that you picked by hand
- ▶ The input space of even a small FIFO is enormous: which values, how many, in which order?
- ▶ PBT turns it around: state a **property** that must hold for *all* inputs, and let a tool pick the inputs
 1. Name the property: “push then pop returns the value”
 2. Describe the valid inputs with a **generator**
 3. Run the property a few hundred times with random inputs
- ▶ Properties about order need an expected answer. A Scala `List` is a fine reference model for a FIFO
- ▶ When a run fails, the input is **shrunk** to the smallest one that still fails

Property-Based Testing in Chisel

- ▶ ScalaCheck picks the inputs, ScalaTest runs the property, chiseltest drives the hardware
- ▶ Property: what goes into a stage comes out of it

```
"A stage" should "pop what was pushed" in {  
  forAll(Gen.choose(0, 255)) { in =>  
    test(new FifoStage(8)) { c =>  
      c.io.push.poke(true.B)  
      c.io.input.poke(in.U)  
      c.clock.step()  
      c.io.push.poke(false.B)  
      c.io.pop.poke(true.B)  
      c.io.output.expect(in.U)  
      c.clock.step()  
      c.io.empty.expect(true.B)  
    }  
  }  
}
```

Example: Order is Preserved

```
val depth = 8
val items = Gen.resize(depth, Gen.listOf(Gen.choose(0, 255)))
"A FIFO" should "pop items in push order" in {
  forAll(items) { list =>
    test(new Fifo(8, depth)) { c =>
      for (item <- list) { // push
        while (c.io.full.peek().litToBoolean) c.clock.step()
        c.io.push.poke(true.B)
        c.io.input.poke(item.U)
        c.clock.step()
        c.io.push.poke(false.B)
      }
      for (item <- list) { // pop
        while (c.io.empty.peek().litToBoolean) c.clock.step()
        c.io.output.expect(item.U)
        c.io.pop.poke(true.B)
        c.clock.step()
        c.io.pop.poke(false.B)
      }
    }
  }
}
```

Generators, Arbitrary and Shrinking

- ▶ A Gen builds a random value: `choose`, `listOf`, `oneOf`, combined with `map/flatMap`
- ▶ `Arbitrary[T]`: the default generator for a type, used when `forAll` is given no generator
- ▶ The default `Arbitrary[Int]` gives negatives and `Int.MaxValue`; `.U` dislikes both, so provide your own

```
implicit val bytes: Arbitrary[List[Int]] =  
  Arbitrary(Gen.resize(8, Gen.listOf(Gen.choose(0, 255))))
```

Generators, Arbitrary and Shrinking

- ▶ A Gen builds a random value: `choose`, `listOf`, `oneOf`, combined with `map/flatMap`
- ▶ `Arbitrary[T]`: the default generator for a type, used when `forAll` is given no generator
- ▶ The default `Arbitrary[Int]` gives negatives and `Int.MaxValue`; `.U` dislikes both, so provide your own

```
implicit val bytes: Arbitrary[List[Int]] =  
  Arbitrary(Gen.resize(8, Gen.listOf(Gen.choose(0, 255))))
```

- ▶ **Shrinking**: after a failure, `ScalaCheck` retries with smaller inputs and reports the smallest one that still fails
- ▶ Shrinking ignores your generator's constraints; `implicit val noShrink: Shrink[Int] = Shrink.shrinkAny` turns it off

Getting Started

1. Add the ScalaTest/ScalaCheck bridge to build.sbt

```
libraryDependencies += "org.scalatestplus" %%  
  "scalacheck-1-18" % "3.2.19.0" % Test
```

2. Import it and mix it into the test class you already have

```
import org.scalacheck.{Arbitrary, Gen}  
import org.scalatestplus.scalacheck._
```

```
class FifoSpec extends AnyFlatSpec  
  with ChiselScalatestTester  
  with ScalaCheckPropertyChecks {  
  "A FIFO" should "..." in {  
    forAll(items) { list => ... }  
  }  
}
```

3. sbt test, as always

4. More or fewer runs: forAll(items, minSuccessful(200))

Continuous Integration (CI)

Tests are only useful if they are run!

- ▶ “It works on my machine” means little: you forgot half the tests, and your Java is a different version anyway

Continuous Integration (CI)

Tests are only useful if they are run!

- ▶ “It works on my machine” means little: you forgot half the tests, and your Java is a different version anyway
- ▶ A CI server checks out every push and every pull request, in a clean environment, and runs `sbt test`
- ▶ The result is a red or green mark on the pull request; a red one cannot be merged
- ▶ So `master` always builds and always passes

Continuous Integration (CI)

Tests are only useful if they are run!

- ▶ “It works on my machine” means little: you forgot half the tests, and your Java is a different version anyway
- ▶ A CI server checks out every push and every pull request, in a clean environment, and runs `sbt test`
- ▶ The result is a red or green mark on the pull request; a red one cannot be merged
- ▶ So `master` always builds and always passes
- ▶ The same machinery can build the documentation, run the slow benchmarks nightly, or program an FPGA
- ▶ Worth it even when you work alone
- ▶ We use GitHub Actions. Alternative: GitLab CI, Jenkins, CircleCI, Travis CI

GitHub Actions in Three Steps

1. Put this in `.github/workflows/ci.yml`

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17
      - uses: sbt/setup-sbt@v1
      - run: sbt test
```

2. Commit and push. The run shows up under the *Actions* tab within a minute
3. In the repository settings, require the check to pass before a pull request can be merged

Lab Session: Priority List

- ▶ A hardware **priority queue**: push key/value pairs in any order, pop them smallest key first
- ▶ Built as a **systolic array**: a chain of identical nodes, each holding one pair

Lab Session: Priority List

- ▶ A hardware **priority queue**: push key/value pairs in any order, pop them smallest key first
- ▶ Built as a **systolic array**: a chain of identical nodes, each holding one pair
- ▶ **Push**: the new pair enters at the front and bubbles towards its place, one node per cycle
- ▶ **Pop**: the pair with the smallest key leaves from the end
- ▶ Each node only talks to its two neighbours, so the clock rate does not depend on the length

Lab Session: Priority List

- ▶ A hardware **priority queue**: push key/value pairs in any order, pop them smallest key first
- ▶ Built as a **systolic array**: a chain of identical nodes, each holding one pair
- ▶ **Push**: the new pair enters at the front and bubbles towards its place, one node per cycle
- ▶ **Pop**: the pair with the smallest key leaves from the end
- ▶ Each node only talks to its two neighbours, so the clock rate does not depend on the length
- ▶ Throughput: one pop every two cycles, after the initial latency
- ▶ Parameters: key width, value width, and number of nodes

Commands and Configuration

Commands

- ▶ IDLE – Do nothing
- ▶ PUSH – Insert a key/value pair at the front of the array
- ▶ POP – Remove the pair with the smallest key from the end
- ▶ PUSH/POP – Both in the same cycle

Commands and Configuration

Commands

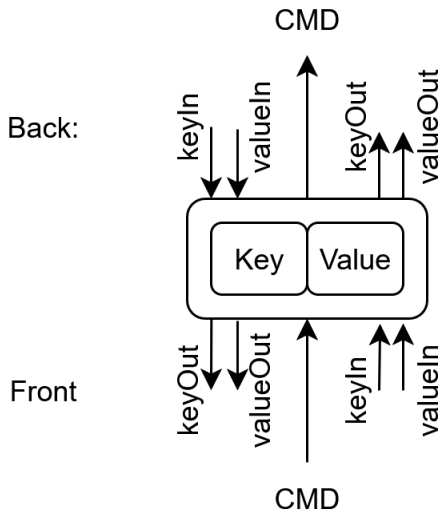
- ▶ IDLE – Do nothing
- ▶ PUSH – Insert a key/value pair at the front of the array
- ▶ POP – Remove the pair with the smallest key from the end
- ▶ PUSH/POP – Both in the same cycle

Parameters

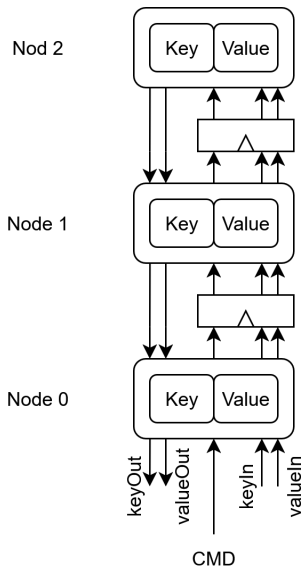
- ▶ keyWidth – Bit width of the key, which decides the order
- ▶ valueWidth – Bit width of the value carried along
- ▶ depth – Number of nodes in the array

Each node is its own module, so it can be unit tested on its own

Systolic Array Priority List: Node



Systolic Array Priority List



Testing the Priority List

Exercise: 3 mins. You have a node module and the full array.
What goes on the test list?

Testing the Priority List

Exercise: 3 mins. You have a node module and the full array.
What goes on the test list?

- ▶ **Unit tests**, one node: holds a pair, compares with its neighbour, forwards the right one, IDLE changes nothing
- ▶ **Integration tests**, the array: push then pop returns the pair, pops come out sorted, push and pop in the same cycle, throughput is one pop per two cycles
- ▶ **Properties**: every pushed pair is popped exactly once, popped keys never decrease, equal keys come out in push order, IDLE cycles change nothing

Testing the Priority List

Exercise: 3 mins. You have a node module and the full array.
What goes on the test list?

- ▶ **Unit tests**, one node: holds a pair, compares with its neighbour, forwards the right one, IDLE changes nothing
- ▶ **Integration tests**, the array: push then pop returns the pair, pops come out sorted, push and pop in the same cycle, throughput is one pop per two cycles
- ▶ **Properties**: every pushed pair is popped exactly once, popped keys never decrease, equal keys come out in push order, IDLE cycles change nothing
- ▶ Which of these need a reference model? A sorted Scala list is enough

Extension: Grouped Values

New requirements

- ▶ A key can have several values: values are tracked in **groups**
- ▶ One PUSH may carry a variable number of values
- ▶ Values pushed with an existing key **merge** into that key's group
- ▶ The maximum group size is a parameter

Extended commands

- ▶ Push($\emptyset$) – No operation
- ▶ Pop($\emptyset$) – Pop the lowest key and its whole group
- ▶ Push(x) – Push x values with a given key
- ▶ Pop(x) – Pop the lowest key group, then push x values with a new key

Summary

- ▶ You wrote ten FIFO tests in three minutes. Write the list before the hardware
- ▶ Red, green, refactor: one test at a time
- ▶ Property-based tests find the inputs you would not have thought of
- ▶ CI runs the tests you would have forgotten, on a machine that is not yours
- ▶ Lab: the priority list, tests first